

Avaliação de Modelos

Francisco Rodrigues

Instituto de Ciências Matemáticas e de Computação

Universidade de São Paulo

francisco@icmc.usp.br

Avaliação

- Não existe técnica de aprendizado de máquina universal, que se saia melhor em qualquer tipo de problema (***No free lunch theorem***).
- Mesmo que um único algoritmo seja escolhido, variações de parâmetros produzem diferentes modelos.
- **Como comparar modelos e parâmetros dos modelos?**
 - **Métricas para classificação:**
 - Taxa de erro
 - Acurácia
 - **Métricas para regressão:**
 - Erro quadrático médio
 - Distância absoluta média

- Taxa de erro de um classificador:

$$E(f) = \frac{1}{n} \sum_{i=1}^n I(y_i \neq f(\mathbf{x}_i))$$

- Onde $I(\cdot)$ é a função indicadora, sendo igual a 1 se a entrada for verdadeira.
- $E(f)$ varia entre zero e um, sendo melhor quando for próximo de zero.

- Taxa de acerto (acurácia)

$$Ac(f) = 1 - E(f) = 1 - \frac{1}{n} \sum_{i=1}^n I(y_i \neq f(\mathbf{x}_i))$$

- Proporção de exemplos classificados corretamente em um conjunto com n objetos.
- Varia entre 0 e 1 e valores próximos de 1 são melhores.

Avaliação

- **Matriz de confusão**
 - Linhas representam classes verdadeiras.
 - Colunas representam classes preditas.
 - Elemento A_{ij} : número de exemplos da classe c_i classificados como pertencentes à classe c_j .
 - Diagonal da matriz: acertos do classificador.
 - Elementos fora da diagonal: erros cometidos.

		Predicted	
		Spam	Non-spam
Actual	Spam	600	300
	Non-spam	100	9000

		Predicted			
		Iris-setosa	Iris-versicolor	Iris-virginica	Σ
Actual	Iris-setosa	100.0 %	0.0 %	0.0 %	50
	Iris-versicolor	0.0 %	88.7 %	6.4 %	50
	Iris-virginica	0.0 %	11.3 %	93.6 %	50
Σ		50	53	47	150

Avaliação

- Problema de duas classes:
 - Tipos de erros:



Avaliação

- Problema de duas classes:
- Medidas de desempenho:

		Classe Predita		
		Positivo	Negativo	
Classe Verdadeira	Positivo	Verdadeiro Positivo (VP)	Falso Negativo (FN)	
	Negativo	Falso Positivo (FP)	Verdadeiro Negativo (VN)	

Sensibilidade: $VP / (VP + FN)$

Precisão: $VP / (VP + FP)$

Especificidade: $VN / (VN + FP)$

Avaliação

- **Problema de duas classes:**
 - Medidas de desempenho:

- **Erro total:**

$$E(f) = \frac{FP + FN}{n}$$

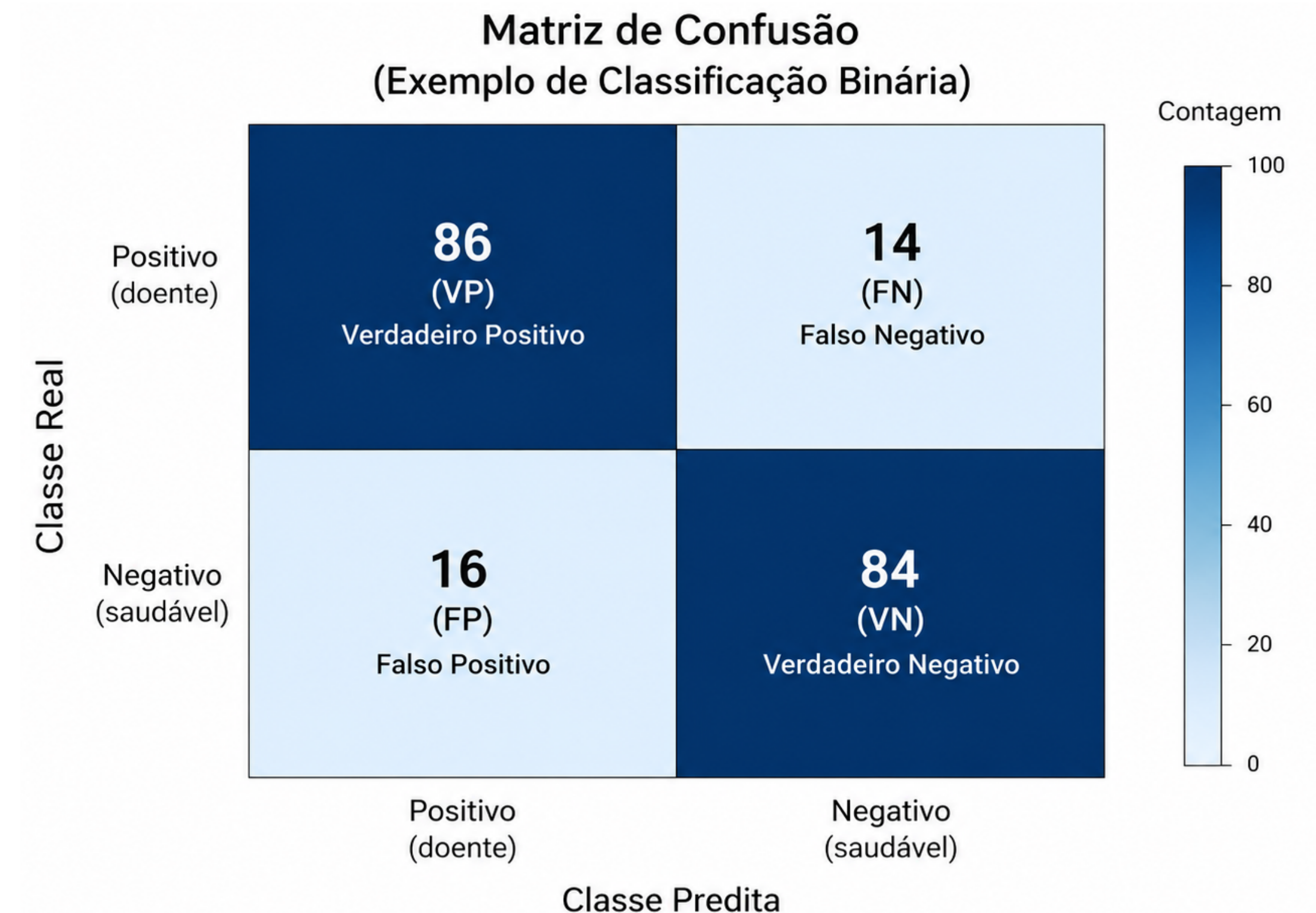
- **Acurácia total:**

$$Ac(f) = \frac{VP + VN}{n}$$

Verdadeiro Positivo (VP)	Falso Negativo (FN)
Falso Positivo (FP)	Verdadeiro Negativo (VN)

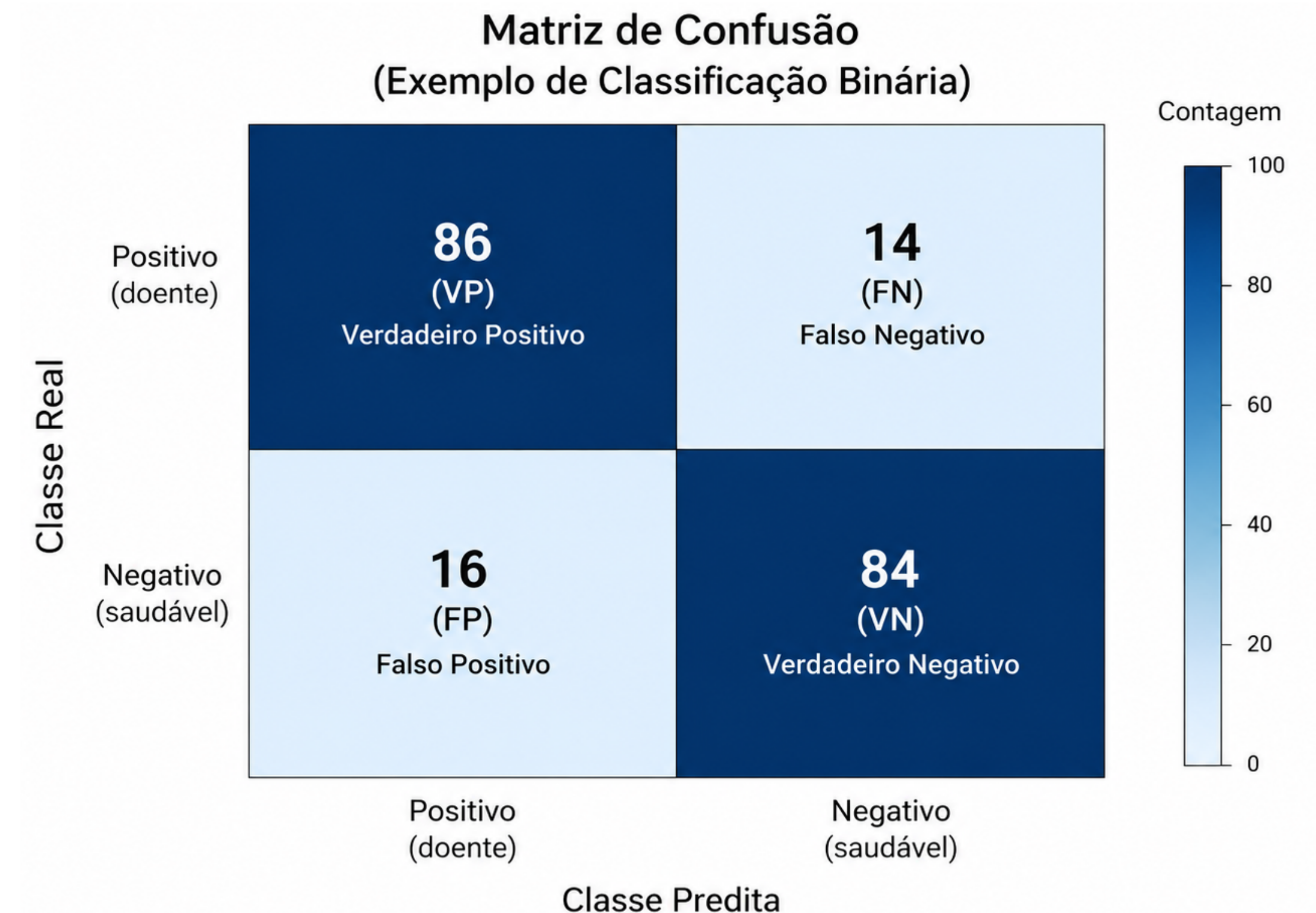
Avaliação

- Problema de duas classes:
- **Falso positivo:** o modelo prevê que o caso é positivo, mas na realidade ele é negativo. Por exemplo, classificar uma pessoa saudável como doente.
- **Falso negativo:** o modelo prevê que o caso é negativo, mas na realidade ele é positivo. Por exemplo, classificar uma pessoa doente como saudável.



Avaliação

- Problema de duas classes:
- **Verdadeiro positivo:** o modelo prevê que o caso é positivo e ele realmente é positivo. Por exemplo, classificar corretamente uma pessoa doente como doente.
- **Verdadeiro negativo:** o modelo prevê que o caso é negativo e ele realmente é negativo. Por exemplo, classificar corretamente uma pessoa saudável como saudável.



Avaliação

- **Problema de duas classes:**

- Medidas de desempenho:

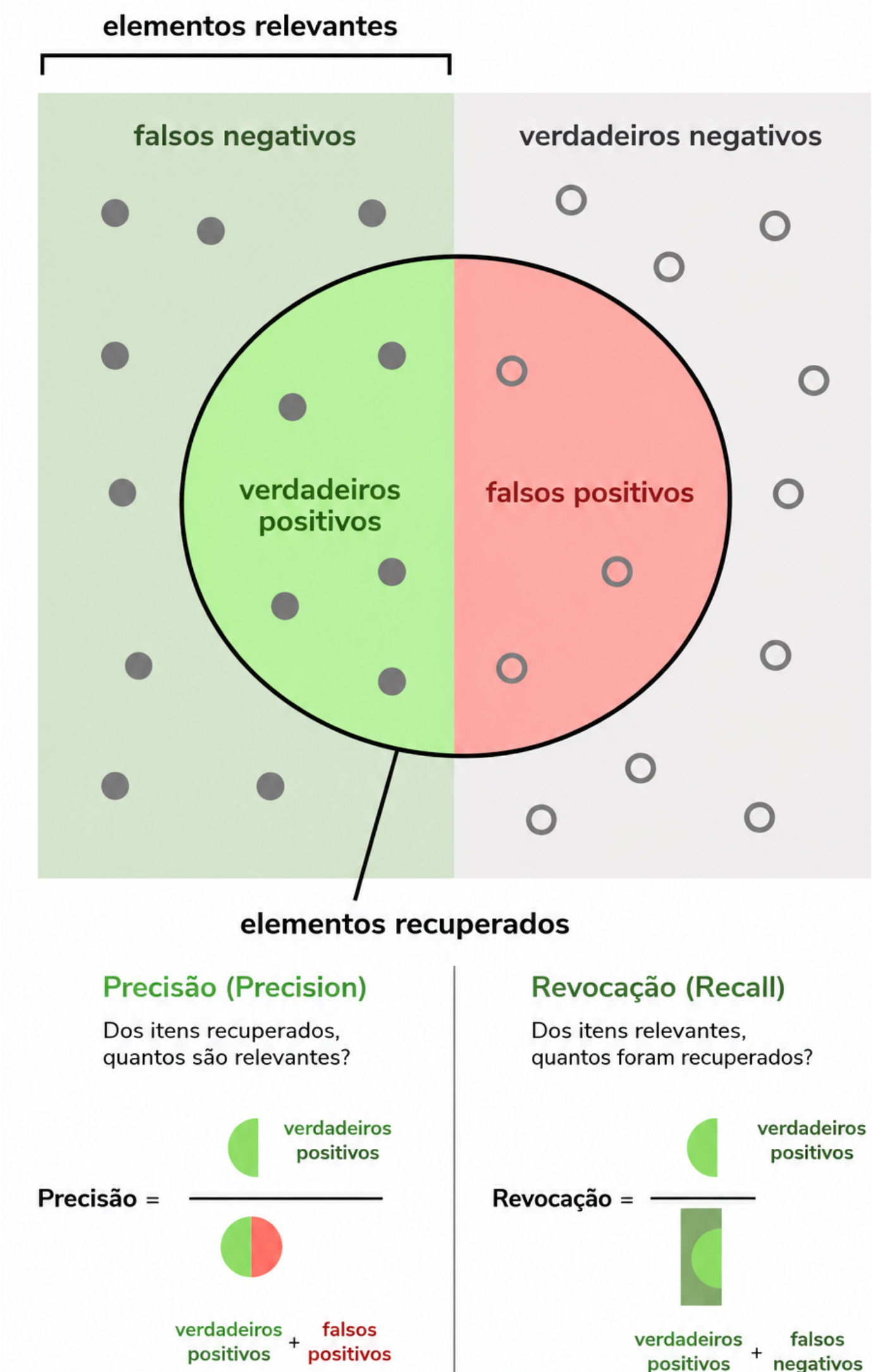
Verdadeiro Positivo (VP)	Falso Negativo (FN)
Falso Positivo (FP)	Verdadeiro Negativo (VN)

- **Precisão (precision):** avalia a quantidade de verdadeiros positivos sobre a soma de todos os valores positivos:

$$\text{Prec}(f) = \frac{VP}{VP + FP}$$

- **Sensibilidade (recall):** avalia a capacidade do método de detectar resultados positivos. Útil quando o custo de falsos negativos é alto.

$$S(f) = \frac{VP}{VP + FN}$$



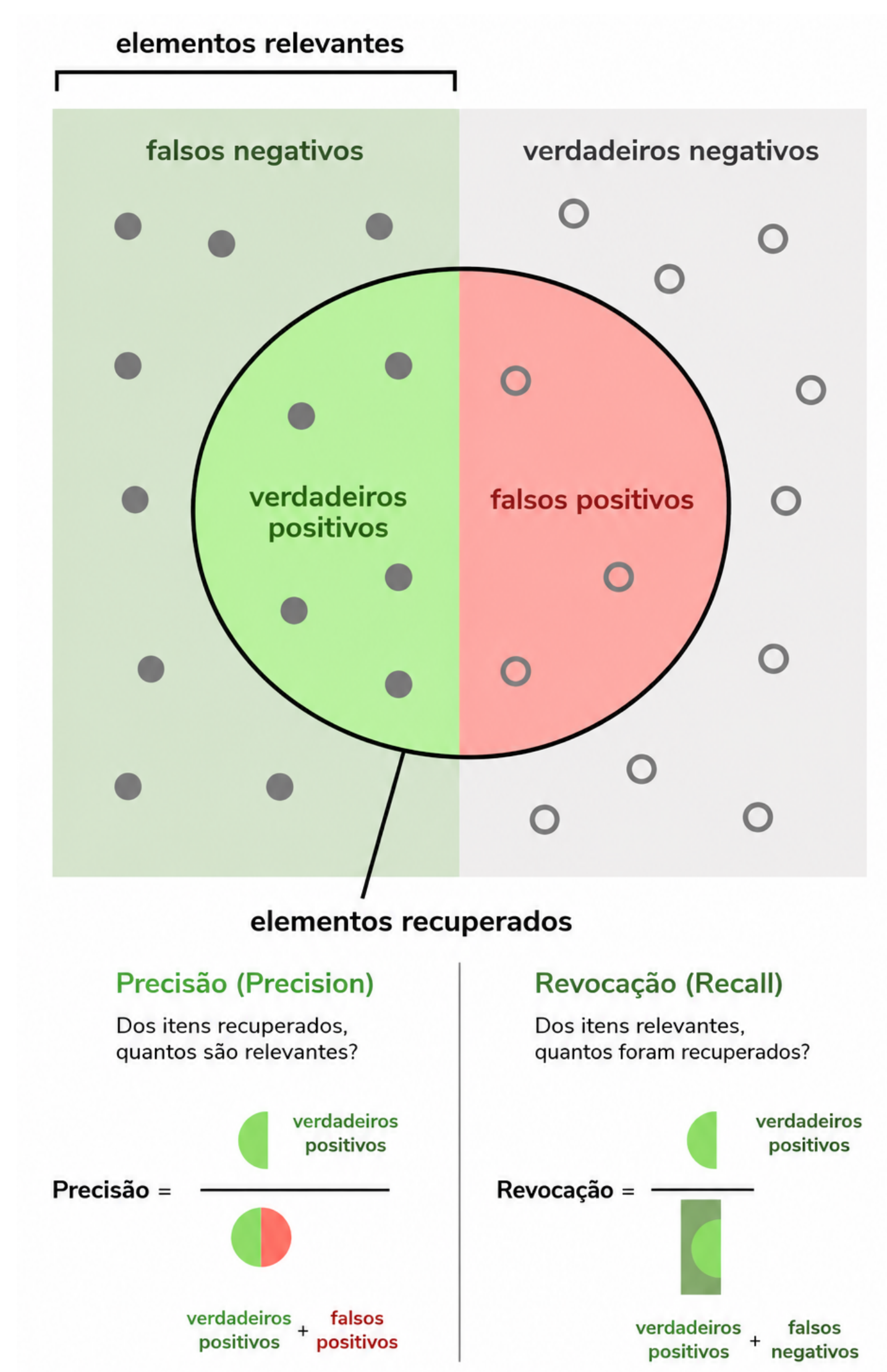
Avaliação

- Problema de duas classes:
- Medidas de desempenho:
 - **Especificidade (true negative rate):** avalia a capacidade do método de detectar resultados negativos.
- **Medida F1:** Média harmônica da precision e recall.

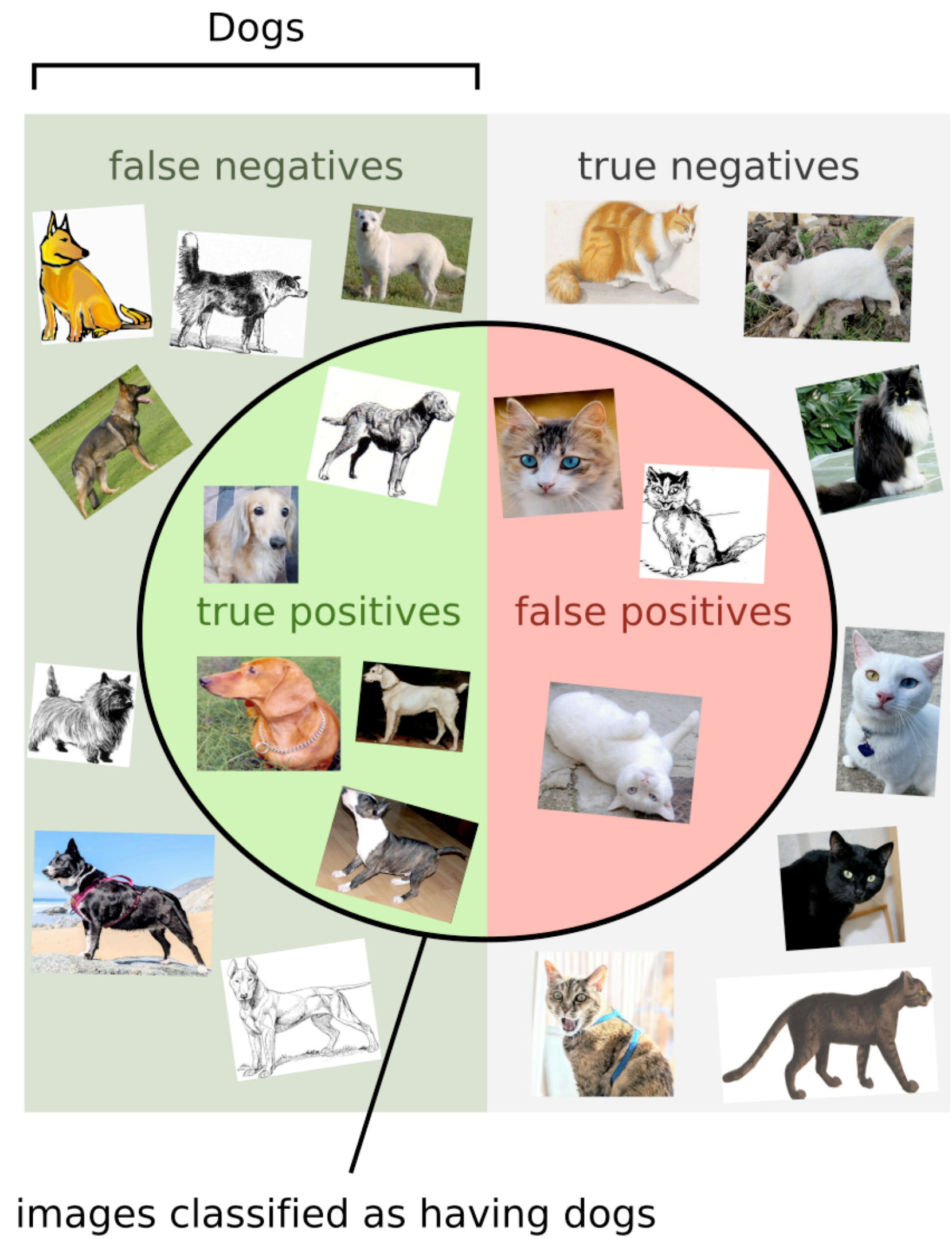
Verdadeiro Positivo (VP)	Falso Negativo (FN)
Falso Positivo (FP)	Verdadeiro Negativo (VN)

$$\text{Esp}(f) = \frac{VN}{VN + FP}$$

$$F1(f) = \frac{2S(f)\text{Prec}(f)}{S(f) + \text{Prec}(f)}$$



Avaliação



$$\text{Precision} = \frac{5 \text{ true pos.}}{8 \text{ total pos.}} \quad \text{Recall} = \frac{5 \text{ true pos.}}{12 \text{ total dogs}}$$

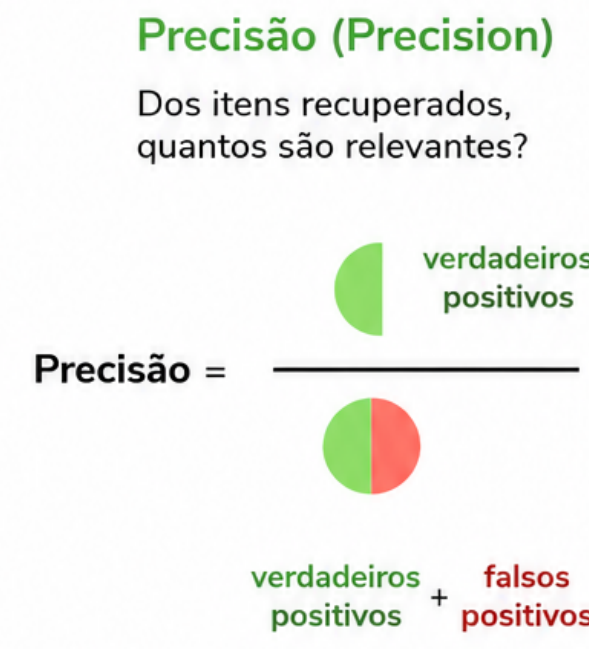
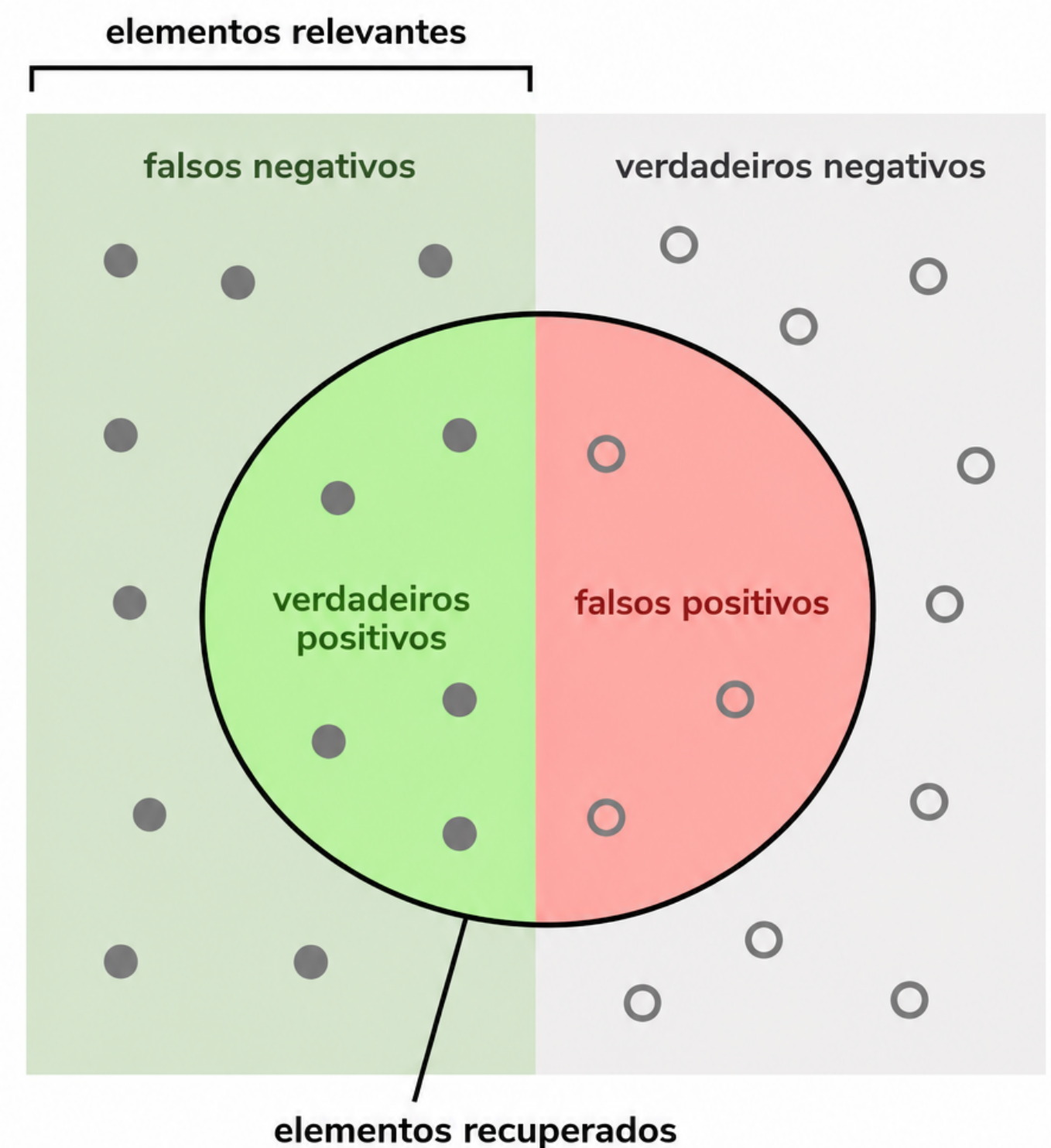
$$\text{Prevalence} = \frac{12 \text{ total dogs}}{22 \text{ total images}}$$

$$\text{Accuracy} = \frac{5 \text{ true pos.} + 7 \text{ true neg.}}{22 \text{ total images}}$$

Avaliação

- Medida F1:
- Desvantagens:
 - Não considera os verdadeiros negativos, que podem ser necessário em alguns cenários.
 - Ele assume a mesma importância da precision e da recall.
 - Não possui simetria. Seu valor pode mudar se rotularmos novamente amostras "positivas" como "negativas" e vice-versa.

Verdadeiro Positivo (VP)	Falso Negativo (FN)
Falso Positivo (FP)	Verdadeiro Negativo (VN)



$$F_1 = \frac{1}{\text{Pr}^{-1} + \text{Re}^{-1}} = \frac{2 \cdot \text{TP}}{2 \cdot \text{TP} + \text{FP} + \text{FN}}$$

True Positive (TP), False Positive (FP), False Negative (FN), Precision (Pr), Recall (Re)

Avaliação

- Problema de duas classes:

- Exemplo:

- **Acurácia:** $Ac(f) = \frac{VP + VN}{n} = \frac{70 + 60}{200} = 0.65$

- Precision (precisão):

$$Prec(f) = \frac{VP}{VP + FP} = \frac{70}{70 + 40} = 0.64$$

- Sensibilidade (recall):

$$S(f) = \frac{VP}{VP + FN} = \frac{70}{70 + 30} = 0.7$$

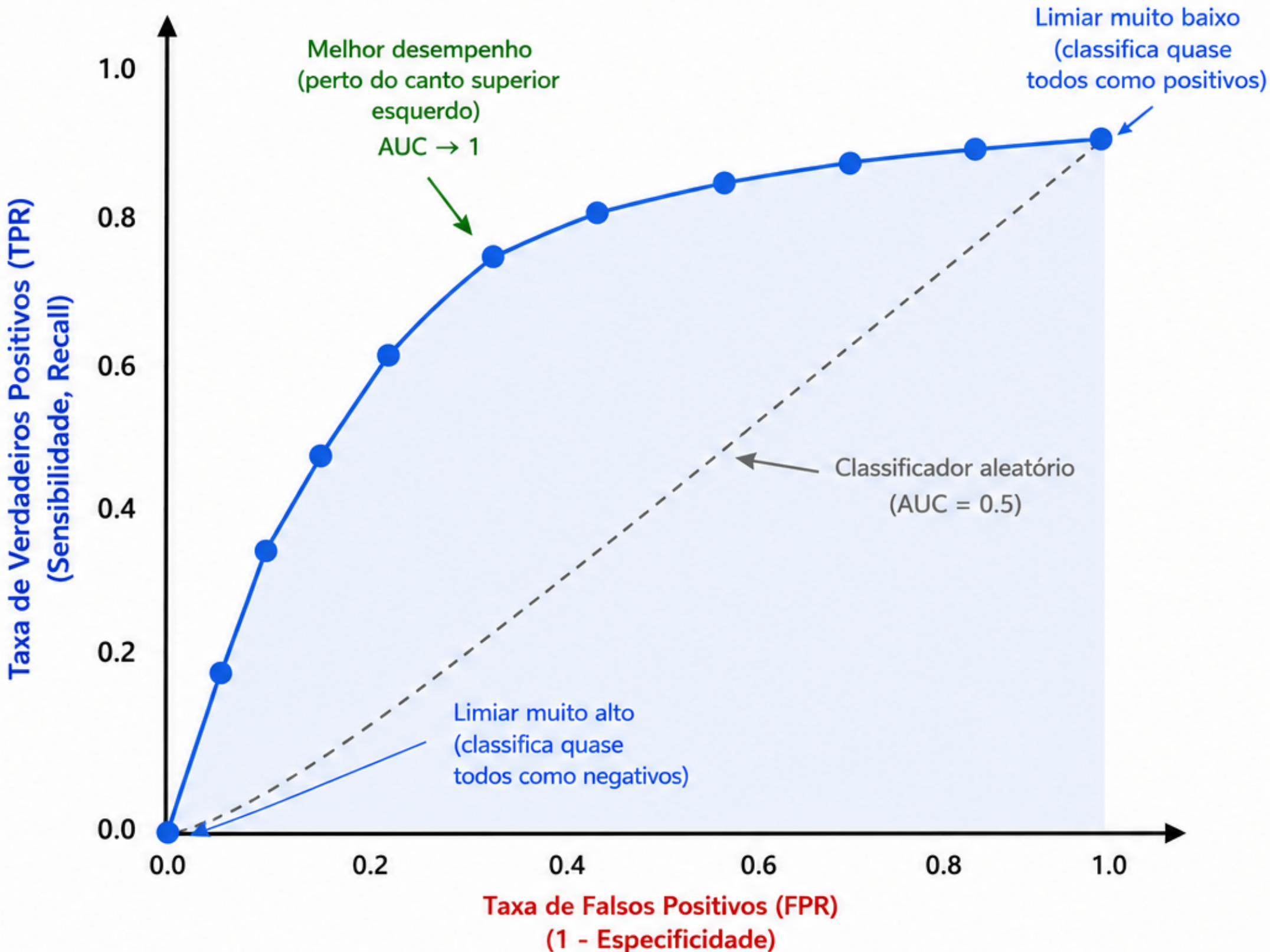
- Especificidade (true negative rate): $Esp(f) = \frac{VN}{VN + FP} = \frac{60}{60 + 40} = 0.60$

		Predito	
		p	n
Verdadeiro	p	VP	FN
	n	FP	VN

		p		n	
		p		n	
Verdadeiro	p	70	30		
	n	40	60		

Curva ROC (Receiver Operating Characteristic)

A curva ROC mostra o desempenho de um classificador para todos os possíveis limiares de decisão.



💡 Quanto maior a área sob a curva (AUC), melhor o desempenho do classificador. A curva ROC é independente do limiar e do desbalanceamento das classes.

1. Eixos

- **Eixo x (FPR):** proporção de negativos classificados incorretamente como positivos (falsos positivos).
- **Eixo y (TPR):** proporção de positivos classificados corretamente (verdadeiros positivos).

2. Pontos da curva

Cada ponto representa o desempenho do classificador para um limiar específico. Ao variar o limiar de decisão de 1 $\rightarrow$ 0, obtemos diferentes combinações de TPR e FPR.

3. Canto superior esquerdo

Região desejável: alta sensibilidade (TPR próxima de 1) e baixa taxa de falsos positivos (FPR próxima de 0).

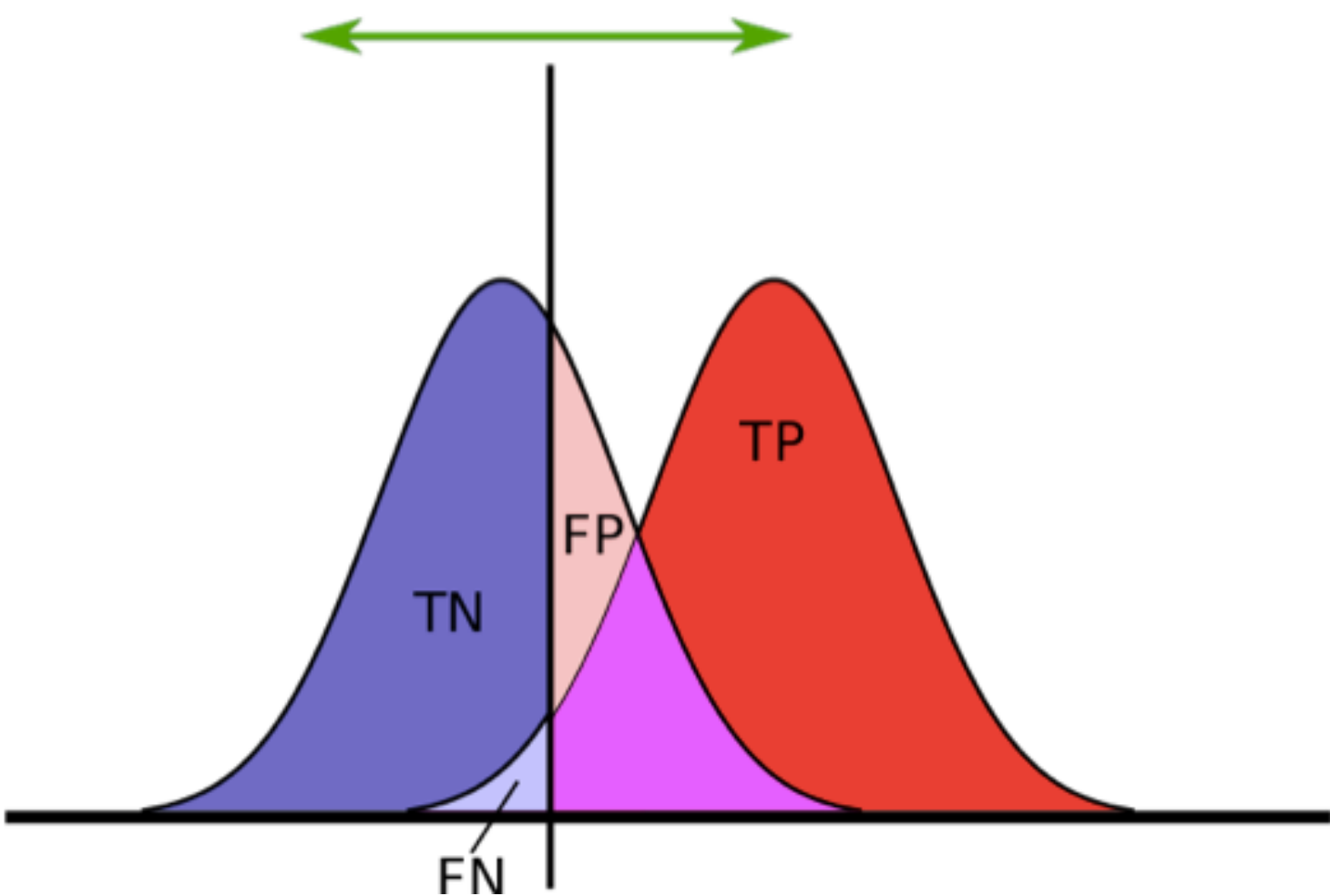
4. Linha tracejada (diagonal)

Representa um classificador aleatório, com desempenho equivalente a chutar. AUC = 0.5.

5. AUC (Área sob a curva)

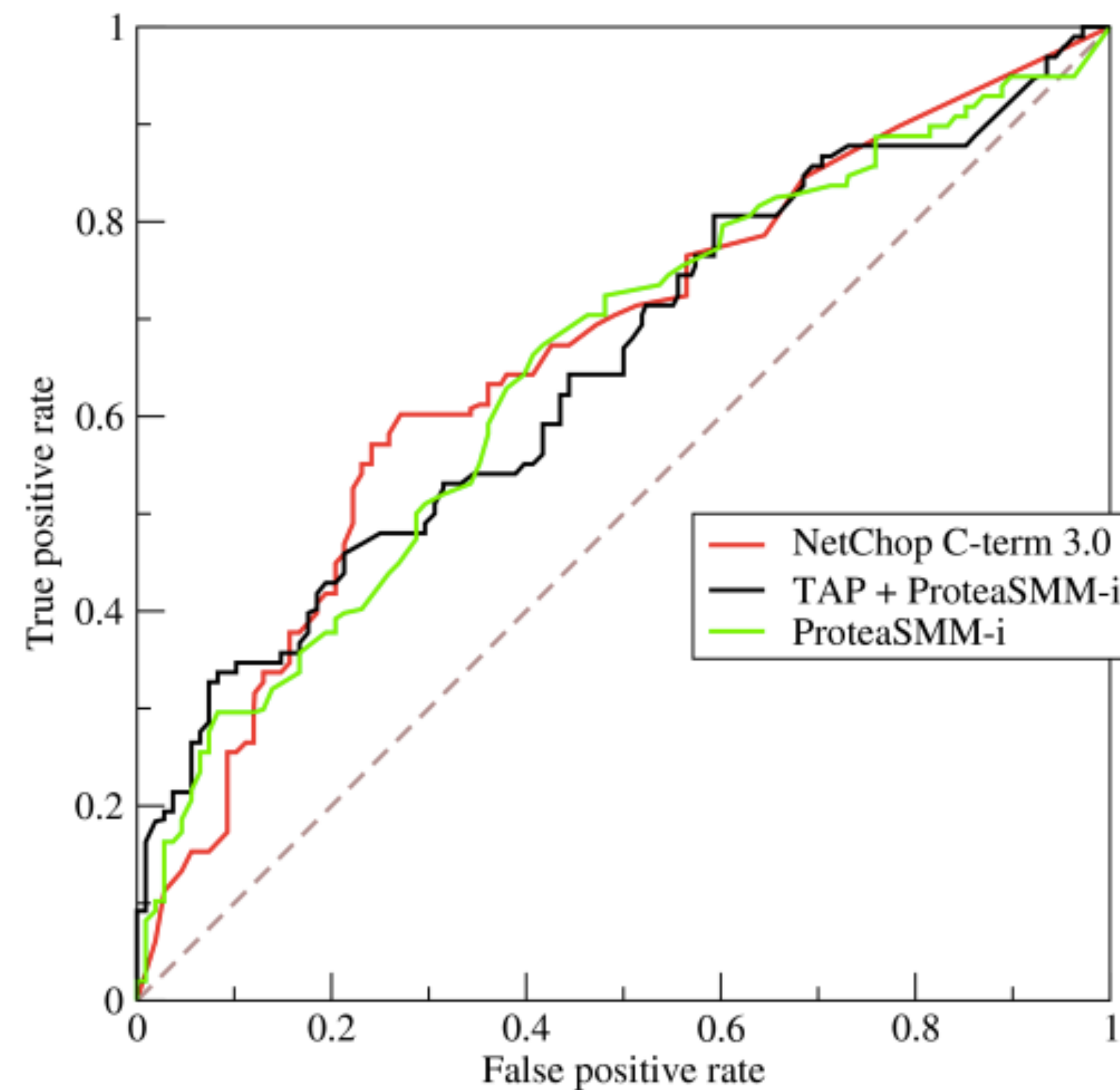
- Mede o desempenho global do classificador:
- AUC = 1.0 $\rightarrow$ classificador perfeito
 - AUC = 0.5 $\rightarrow$ desempenho aleatório
 - AUC < 0.5 $\rightarrow$ pior que aleatório

		Detectada	
		Sim	Não
Real	Sim	Verdadeiro Positivo (VP)	Falso Negativo (FN)
	Não	Falso Positivo (FP)	Verdadeiro Negativo (VN)



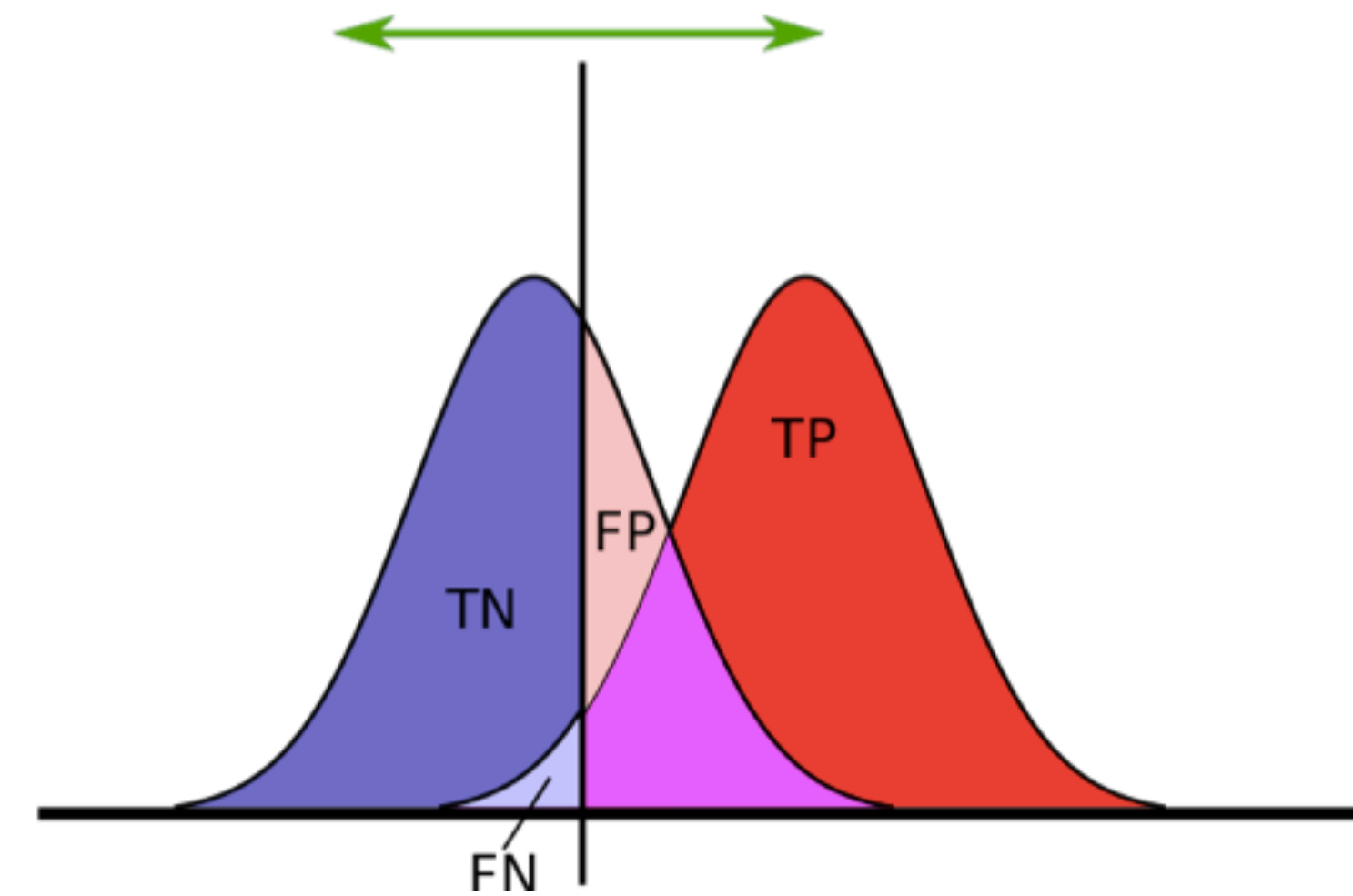
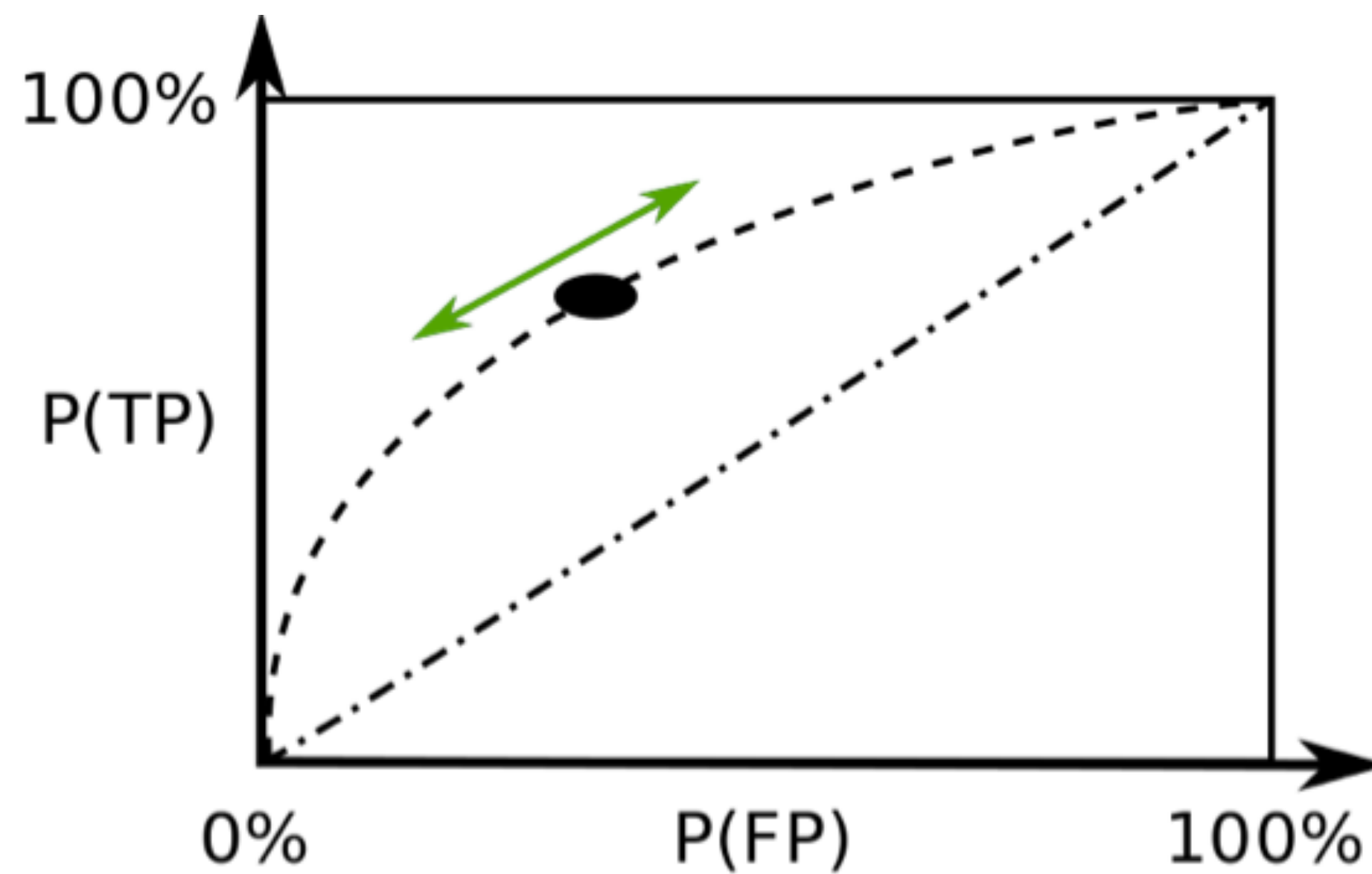
Avaliação

- Problema de duas classes:
 - Curva ROC (Receiving Operating Characteristics):



Avaliação

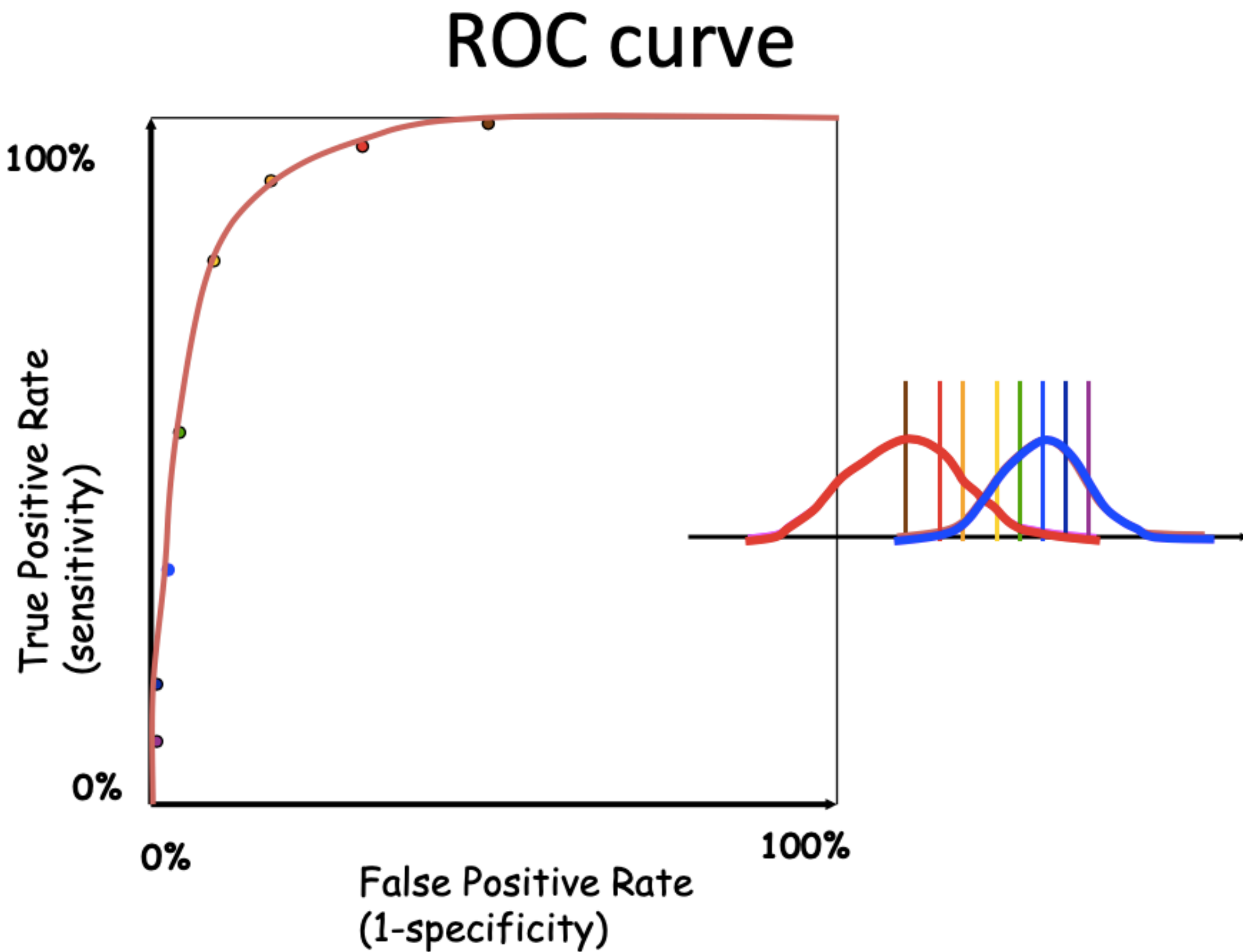
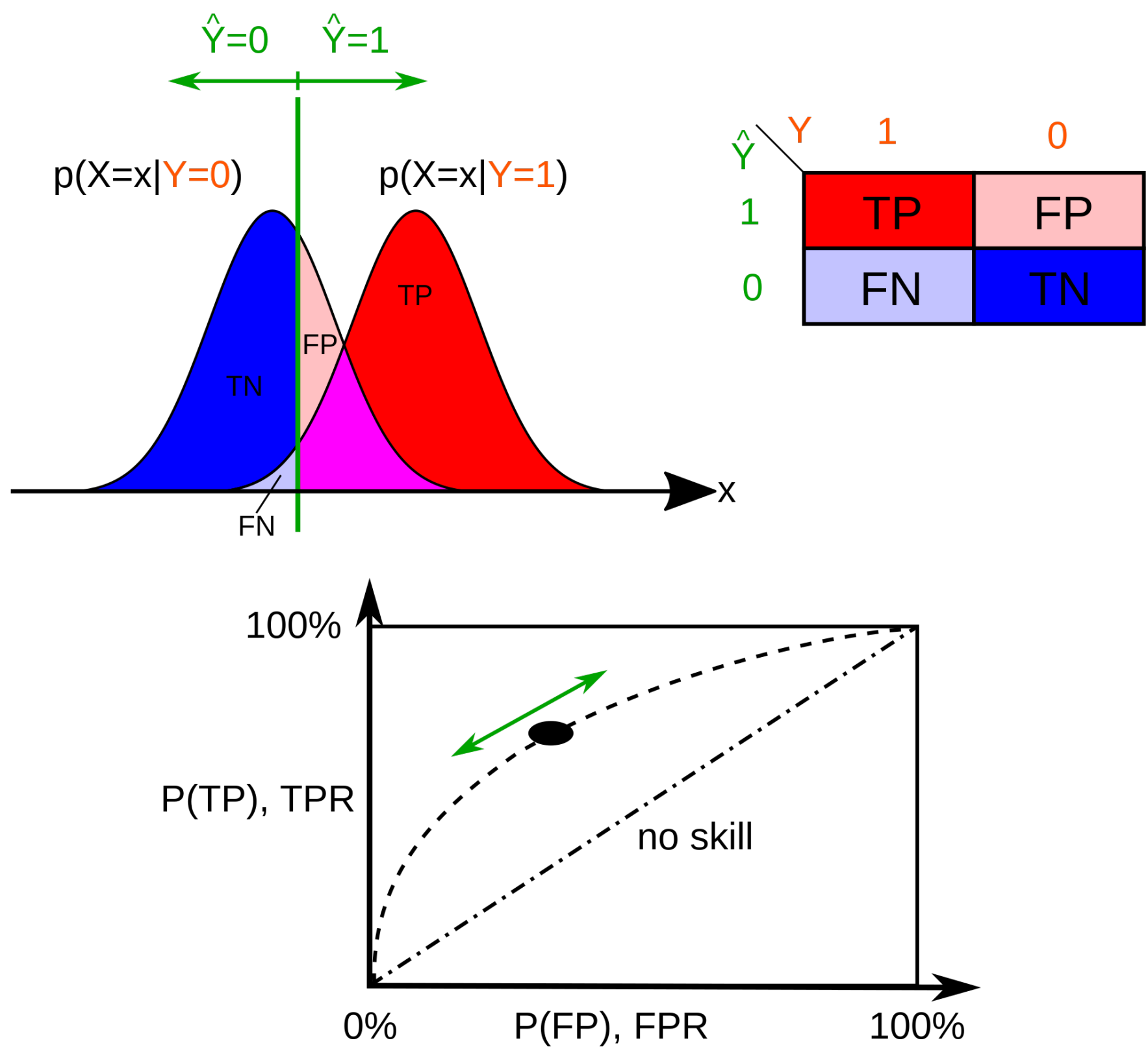
- Problema de duas classes:
 - Curva ROC (Receiving Operating Characteristics):



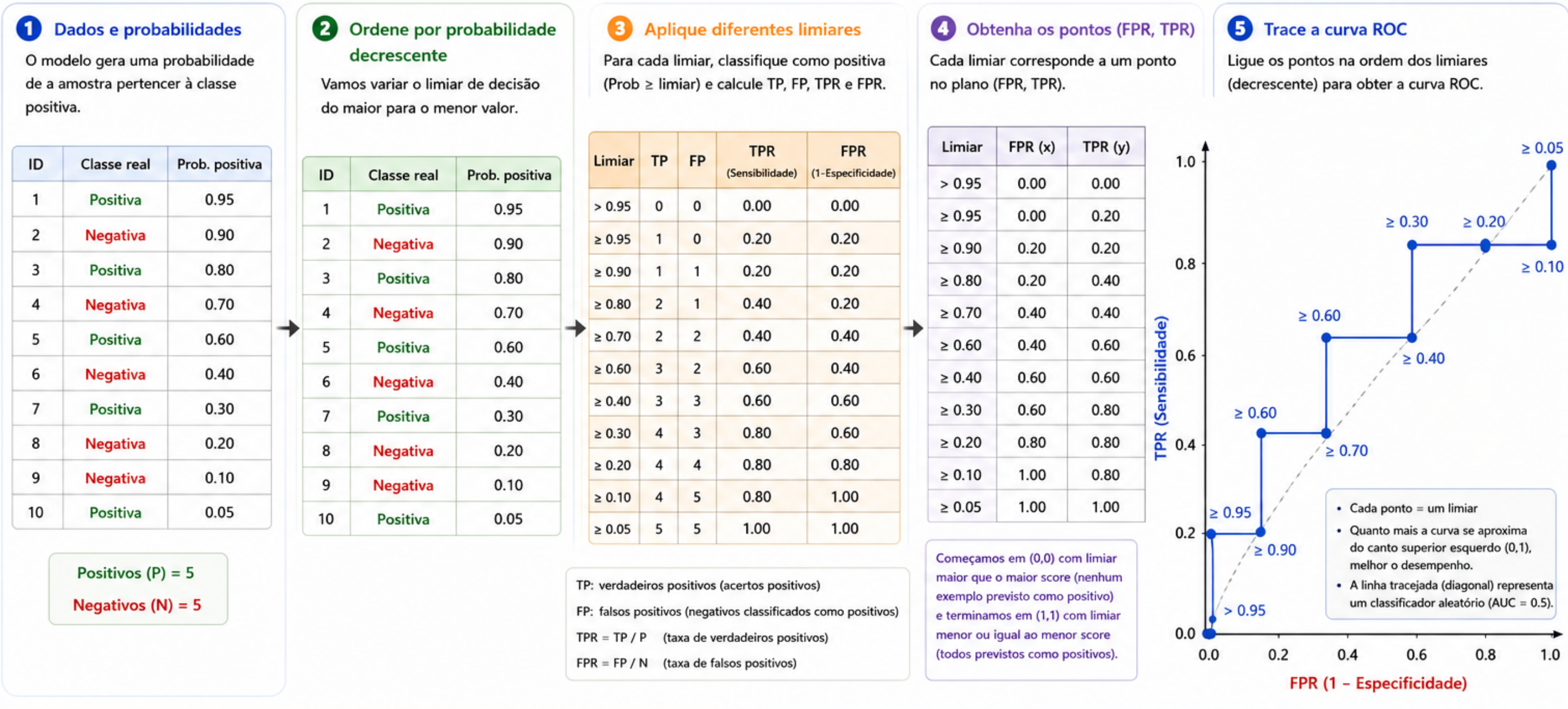
TP	FP
FN	TN

Avaliação

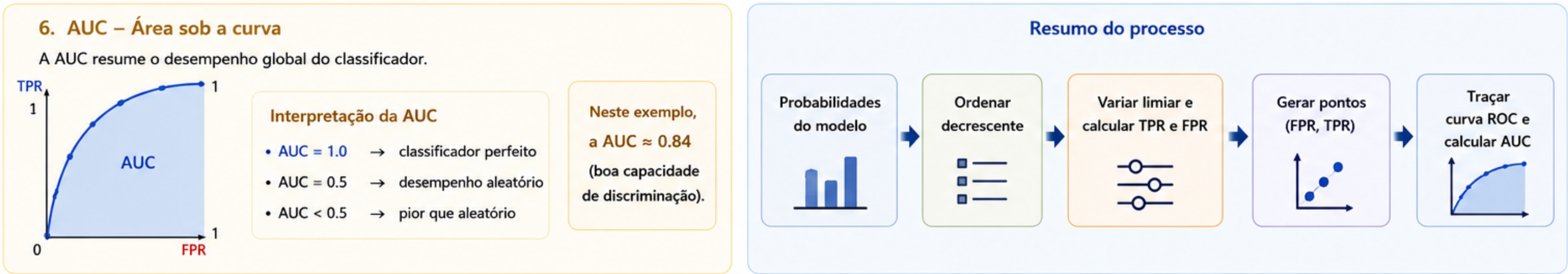
- Problema de duas classes:
 - Curva ROC (Receiving Operating Characteristics):



Construção da Curva ROC – Exemplo Passo a Passo

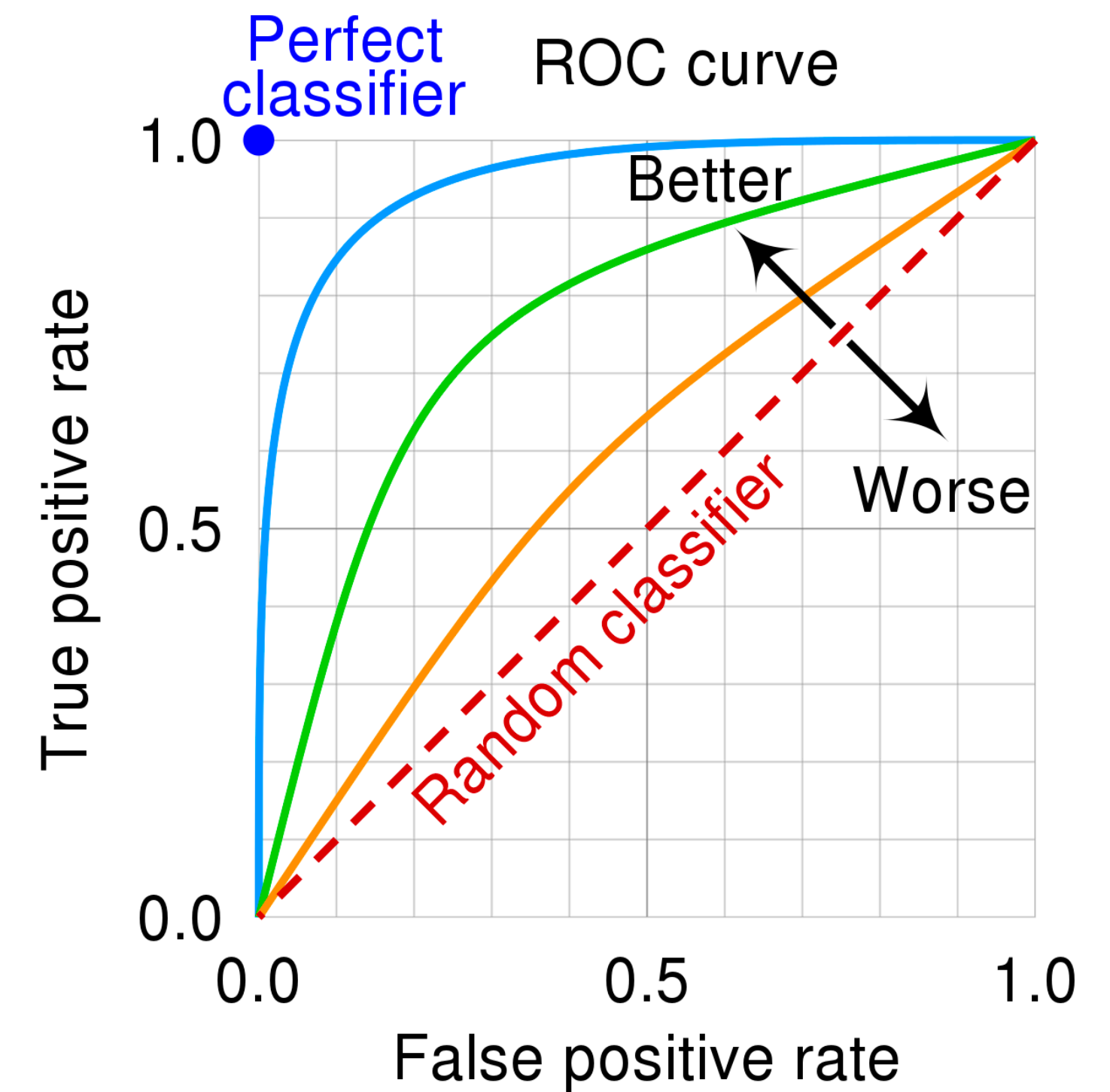
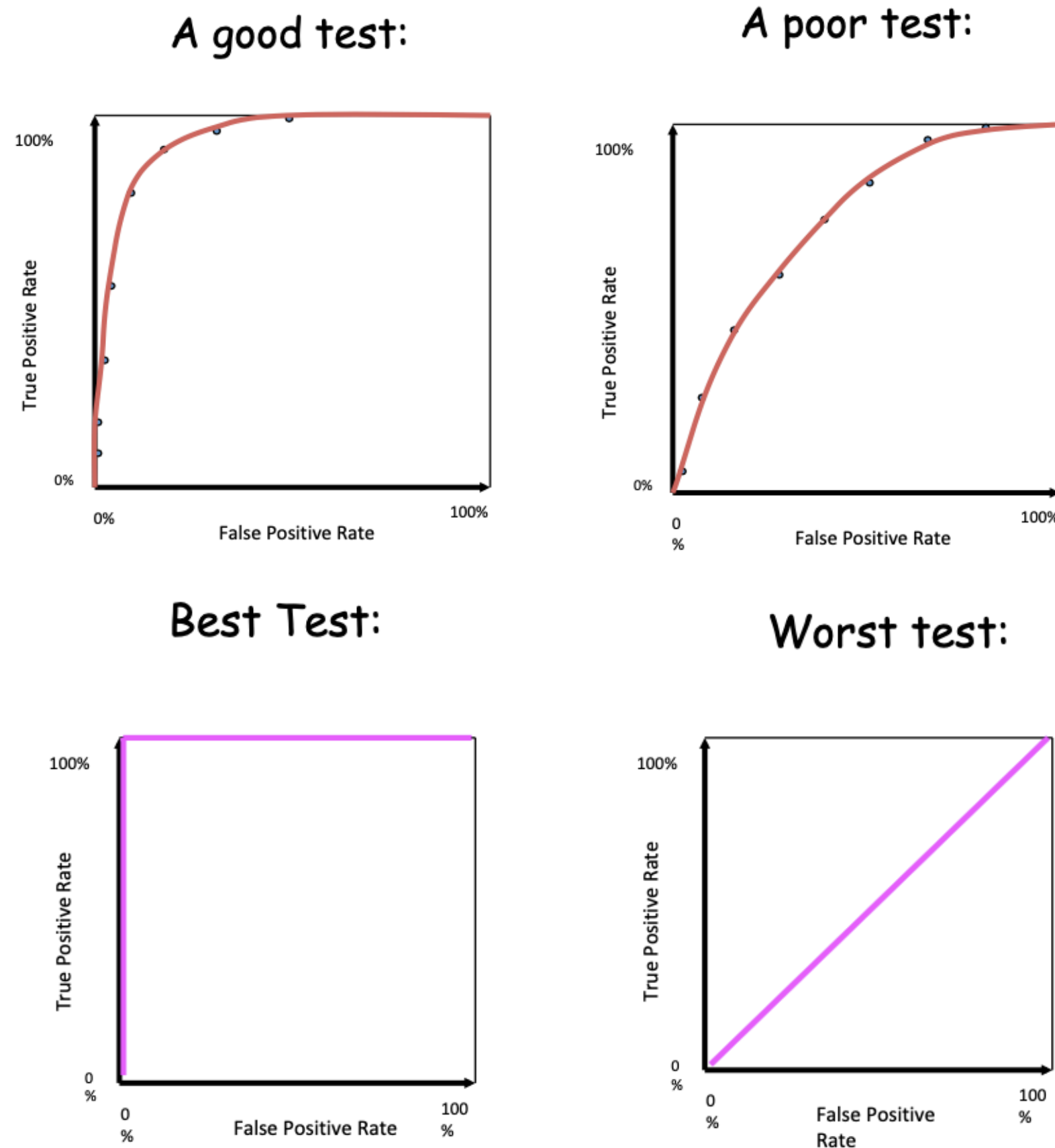


model.predict_proba(testX)



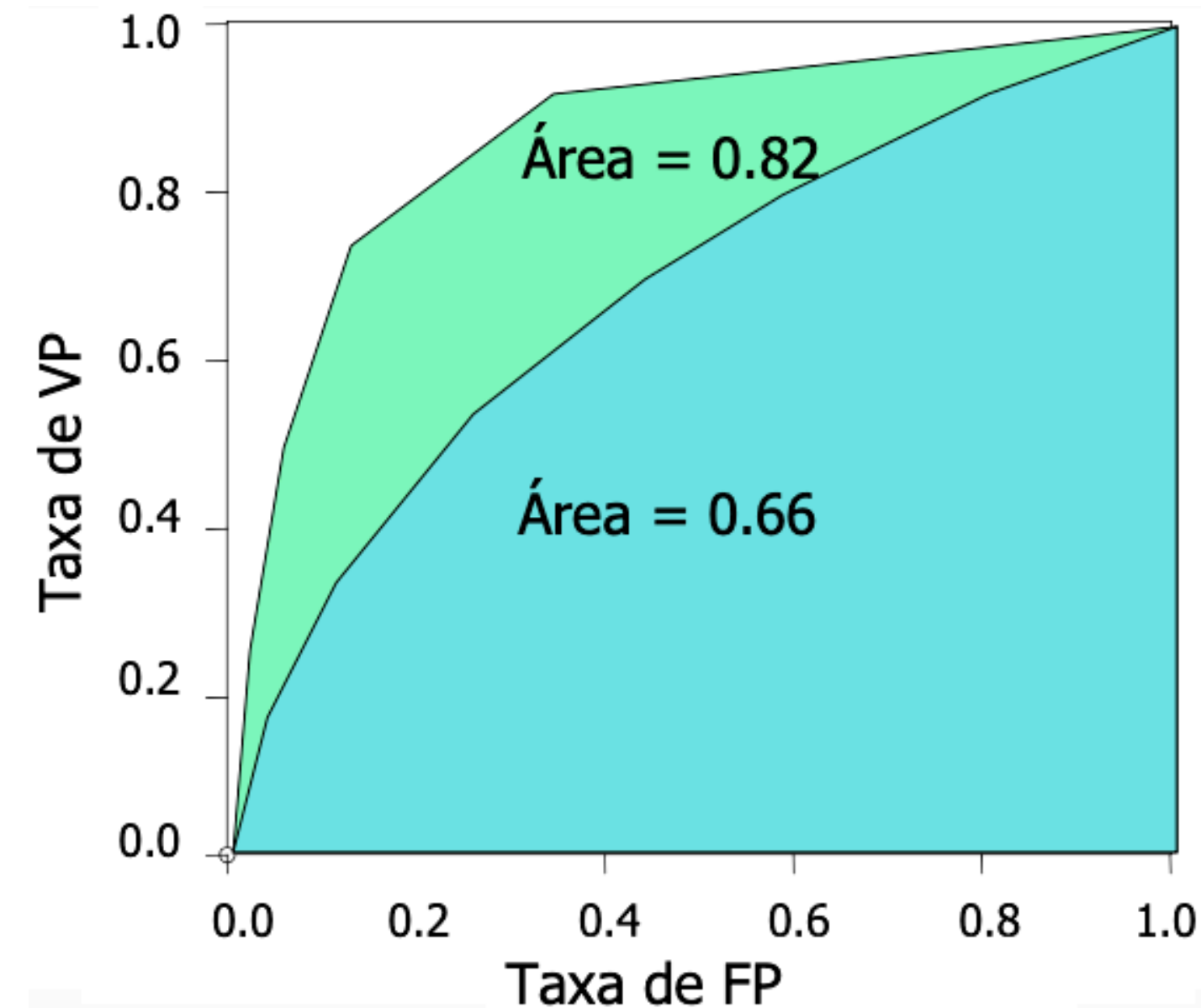
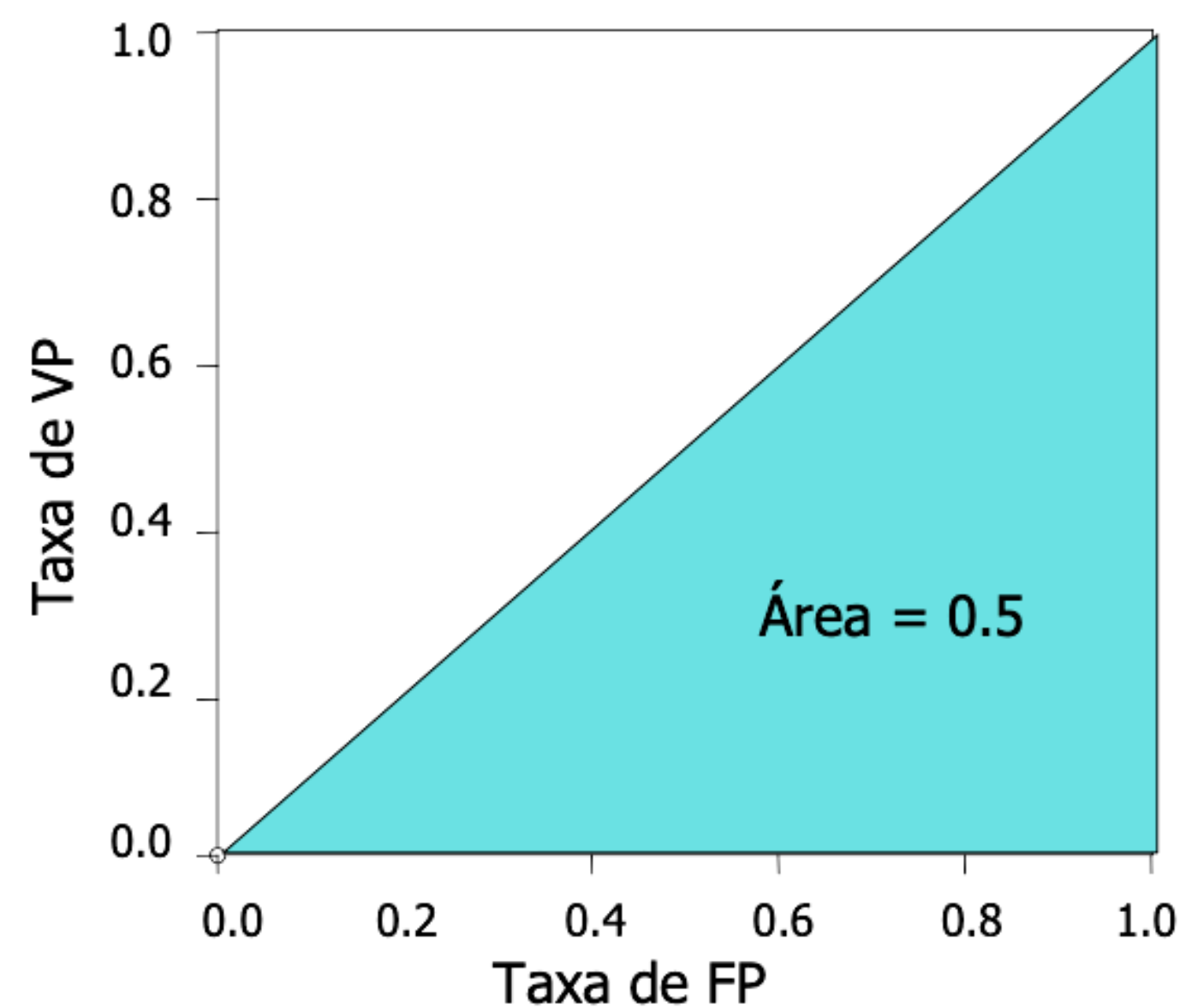
Avaliação

- Problema de duas classes:
- Curva ROC (Receiving Operating Characteristics):

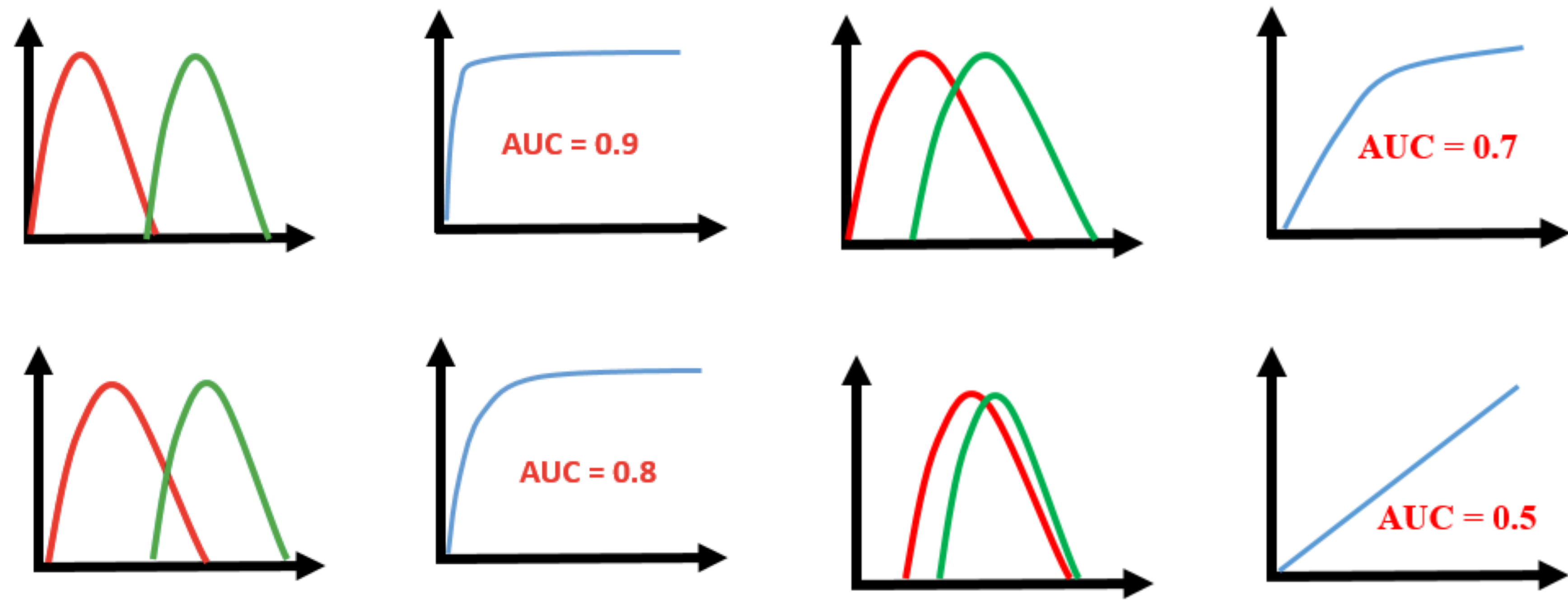


Avaliação

- Problema de duas classes:
 - Curva ROC (Receiving Operating Characteristics):
 - Área sob a curva ROC:
 - Produz valores no intervalo $[0,1]$
 - Valores mais próximos de 1 são considerados melhores.



Avaliação

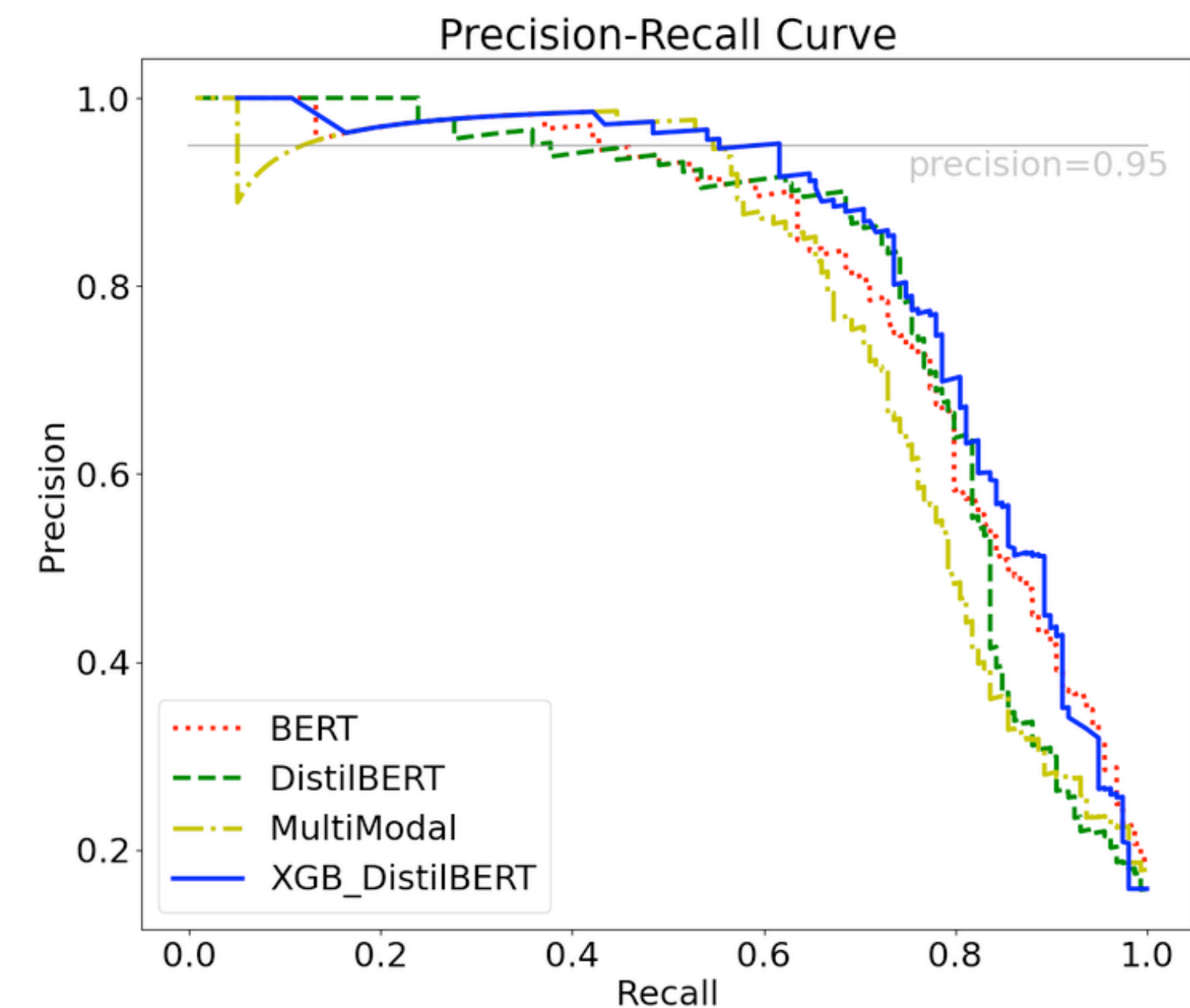


- **Propriedades da Curva ROC:**
- Permite realizar medidas de desempenho independentes do limiar de classificação e de custos associados às classificações incorretas e distribuição das classes.
 - Uso de diferentes limiares representa maior ou menor ênfase à classe positiva.
 - Taxa de erro/acerto é bastante sensível a desbalanceamentos (ex. conjunto com 90 + e 10 -, taxa de acerto de 0,90 não necessariamente indica bom desempenho preditivo)
- **Desvantagem:** análise originalmente limitada a classificação binária.

$$\text{Prec}(f) = \frac{VP}{VP + FP} \quad S(f) = \frac{VP}{VP + FN}$$

- **Precision-Recall**

- Um gráfico ou curva de Precision-Recall é uma ferramenta de avaliação de modelos que plota a precisão no eixo vertical (Y) em função do recall no eixo horizontal (X).
- A curva Precision-Recall não substitui necessariamente a ROC, mas fornece uma visão complementar, especialmente quando o interesse principal está na qualidade das predições positivas.



$$\text{Prec}(f) = \frac{VP}{VP + FP} \quad S(f) = \frac{VP}{VP + FN}$$

- **Precision-Recall**

- **É mais adequada para dados desbalanceados:** Quando a classe positiva é rara, como em diagnósticos médicos, detecção de fraudes ou identificação de transtornos, a curva ROC pode dar uma impressão excessivamente otimista. A curva Precision-Recall mostra melhor o desempenho do modelo na classe que realmente interessa.
- **Foca diretamente na classe positiva:** Ela avalia a capacidade do modelo de encontrar corretamente os casos positivos e, ao mesmo tempo, verificar se as predições positivas são confiáveis.
- **Mostra o compromisso entre precisão e revocação:** A curva permite observar a troca entre encontrar mais casos positivos e evitar falsos positivos. Em geral, ao aumentar o recall, a precisão pode diminuir.
- **Ajuda na escolha do limiar de decisão:** A curva permite escolher um ponto de operação adequado para o modelo, dependendo do objetivo do problema: maior precisão, maior recall ou um equilíbrio entre os dois.
- **É útil quando falsos positivos têm custo alto:** Em algumas aplicações, classificar incorretamente um caso negativo como positivo pode gerar custos financeiros, clínicos ou operacionais. A curva ajuda a avaliar esse tipo de erro.

Avaliação

- O R^2 mede a proporção da variabilidade da variável dependente (aquela que queremos prever) que é explicada pelo modelo.
- Em outras palavras, ele quantifica o quanto o modelo consegue reduzir a incerteza em relação a simplesmente usar a média dos dados.
 - Se $R^2 = 1$, o modelo explica perfeitamente toda a variabilidade dos dados.
 - Se $R^2 = 0$, o modelo não explica nada além do que já seria explicado pela média.
 - Se $R^2 < 0$, significa que o modelo está pior do que usar apenas a média como predição.

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

- **Limitações**

- Não avalia causalidade: um R^2 alto não significa que existe relação causal entre preditores e resposta.
- Pode enganar em modelos não lineares: modelos mal especificados podem ter R^2 alto mas não capturar a estrutura real dos dados.
- Sensível ao número de variáveis: adicionar mais variáveis ao modelo sempre tende a aumentar (ou no mínimo manter) o R^2 , mesmo que elas não sejam relevantes.
- Por isso, muitas vezes se usa o R^2 ajustado, que penaliza o excesso de variáveis.
- Isso pode levar à superestimação da qualidade do modelo, especialmente em regressões com muitas variáveis.

R^2 ajustado

$$\bar{R}^2 = 1 - \frac{n - 1}{n - p - 1} (1 - R^2)$$

- onde p representa o número de variáveis explicativas mais a constante.
- Se você adiciona uma variável relevante, o R^2 ajustado tende a aumentar.
- Se você adiciona uma variável irrelevante, o R^2 ajustado tende a diminuir.
- Ele pode até ser negativo, indicando que o modelo está pior que simplesmente usar a média.

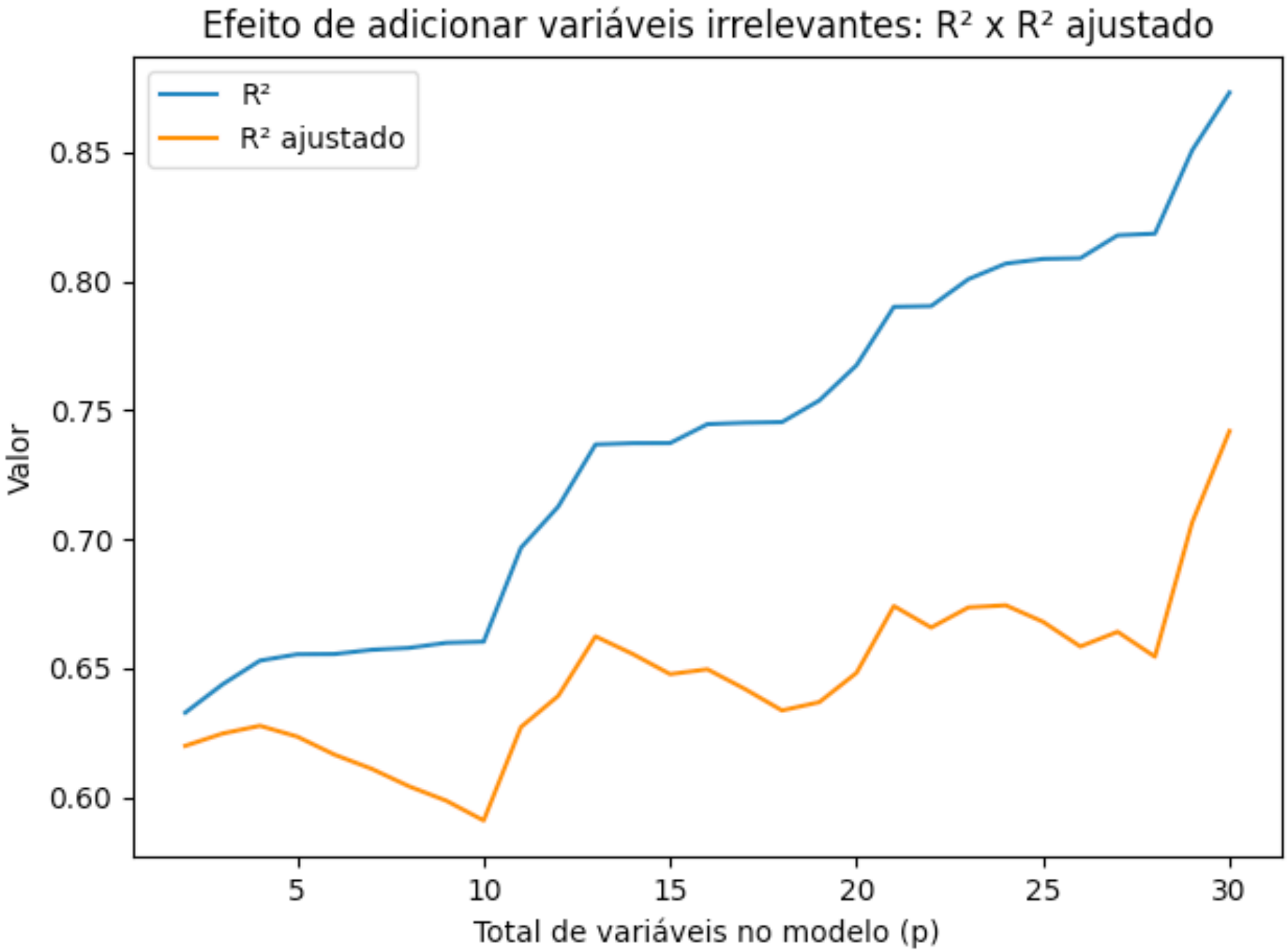
R^2 ajustado

$$\bar{R}^2 = 1 - \frac{n-1}{n-p-1} (1 - R^2)$$

- Se a variável traz informação relevante, o ganho em R^2 é suficiente para compensar a penalização pelo aumento do número de parâmetros, e o R^2 ajustado sobe.
- Se a variável não acrescenta explicação real (ou seja, o aumento em R^2 é pequeno ou nulo), o efeito da penalização domina, e o R^2 ajustado diminui.

R² ajustado

$$\bar{R}^2 = 1 - \frac{n - 1}{n - p - 1} (1 - R^2)$$



```
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt

# Reproducibility
rng = np.random.default_rng(7)

# --- Simulation settings ---
n = 60          # number of samples
p_informative = 2  # true predictors
max_noise = 28    # number of additional noise predictors to try incrementally
sigma = 1.5      # noise std in the response

# Generate informative features
X_inf = rng.normal(size=(n, p_informative))

# True coefficients for informative predictors
beta_true = np.array([2.0, -1.0])

# Generate response with noise
y = X_inf @ beta_true + rng.normal(scale=sigma, size=n)

# Generate a big pool of noise features (independent of y)
X_noise_pool = rng.normal(size=(n, max_noise))

rows = []
model = LinearRegression(fit_intercept=True)

for k in range(0, max_noise + 1):
    # Build the design matrix: informative + first k noise columns
    if k == 0:
        X = X_inf
    else:
        X = np.hstack([X_inf, X_noise_pool[:, :k]])

    p = X.shape[1] # number of predictors (excluding intercept by construction)
    model.fit(X, y)
    r2 = model.score(X, y)

    # Adjusted R²: 1 - (1 - R²)*(n - 1)/(n - p - 1)
    # Only defined if n - p - 1 > 0; otherwise set to NaN.
    denom = (n - p - 1)
    if denom > 0:
        r2_adj = 1 - (1 - r2) * (n - 1) / denom
    else:
        r2_adj = np.nan

    rows.append({
        "Total de variáveis (p)": p,
        "R²": r2,
        "R² ajustado": r2_adj
    })

df = pd.DataFrame(rows)
print(df.round(4))

# Plot the trajectories
plt.figure()
plt.plot(df["Total de variáveis (p)"], df["R²"], label="R²")
plt.plot(df["Total de variáveis (p)"], df["R² ajustado"], label="R² ajustado")
plt.xlabel("Total de variáveis no modelo (p)")
plt.ylabel("Valor")
plt.title("Efeito de adicionar variáveis irrelevantes: R² x R² ajustado")
plt.legend()
plt.tight_layout()
plt.show()
```

<https://colab.research.google.com/drive/19HxHWusGWpE1mKGcW6C1PmAVrf0fLdf2?usp=sharing>

Material adicional

https://scikit-learn.org/stable/modules/model_evaluation.html



3.3. Metrics and scoring: quantifying the quality of predictions

There are 3 different APIs for evaluating the quality of a model's predictions:

- **Estimator score method:** Estimators have a `score` method providing a default evaluation criterion for the problem they are designed to solve. This is not discussed on this page, but in each estimator's documentation.
- **Scoring parameter:** Model-evaluation tools using `cross-validation` (such as `model_selection.cross_val_score` and `model_selection.GridSearchCV`) rely on an internal *scoring* strategy. This is discussed in the section [The scoring parameter: defining model evaluation rules](#).
- **Metric functions:** The `sklearn.metrics` module implements functions assessing prediction error for specific purposes. These metrics are detailed in sections on [Classification metrics](#), [Multilabel ranking metrics](#), [Regression metrics](#) and [Clustering metrics](#).

Finally, [Dummy estimators](#) are useful to get a baseline value of those metrics for random predictions.